先进编译实验室
Advanced Compiling Laboratory
深度学习
编译器实战
--以Triton为例

编译实战培训班

讲师团介绍
先进编译实验室
ADVANCED COMPILING LABORATORY
李帅龙
• 先进编译实验室高性能计算部部长
• FlagGems&FlagTree开源项目贡献者
• 主研算子开发、性能优化
李嘉楠
• 先进编译实验室副主任
• 主研人工智能编译器、编译优化
• 多项开源AI项目负责人
• 2025智源大会Tech Tutorials技术分享者
王磊
• 先进编译实验室研究员
• B站《AI实操优化-Triton编译器》主讲人
• 主研深度学习编译优化
公众号：先进编译实验室

课程目录

- 课程环境配置&Triton与PyTorch
- Triton Kernel性能分析工具
- Triton语言特征详解及关键参数优化
- Triton编译器及Pass管理器
- Triton关键属性Layout介绍
- 编译优化实践(一)：代数变换pass添加
- 编译优化实践(二)：冗余去除pass添加

第二讲课程回顾

1、掌握TorchBench模型基准测试框架的使用

使用Torchbench运行模型生成Triton Kernel和profiler追踪文件

2、掌握编译模块性能的研究分析方法

使用-p选项单独研究每个编译模块的性能，观察终端输出信息的含义

3、掌握profiler追踪文件可视化查看分析方法

使用谷歌（或EDGE）浏览器查看整网和-p生成的.json profiler文件，观察各个可视化条目的含义并进行对比

4、掌握torch.profiler接口的使用方法

使用torch.profiler接口单独分析Kernel，尝试不同的参数选项功能

5、(可选) 了解其他分析工具HTA、Nsight Systems、Nsight Compute、Triton Proton等的安装和使用

先进编译实验室 Advanced Compilation Laboratory

第三讲

Triton 语言特征详解及关键参数优化

为什么进行Triton算子开发

Triton作为一个高效的编译器和编译框架，旨在通过优化和并行化来提高计算性能，特别是在深度学习和其他数值计算密集型应用中，Triton算子开发可以提供较好的性能表现。其原因包括：

（1）并行化和向量化：Triton能够有效地利用GPU的并行计算能力，将计算任务分配给多个线程，从而加速计算过程。此外，Triton支持向量化，可以减少内存使用并提高计算速度。

（2）低级优化：Triton允许开发者编写接近硬件的低级代码，这使得开发者可以手动优化内存访问和计算模式，以适应特定的硬件架构。

（3）内存管理： Triton提供了精细的内存管理功能，如共享内存、内存复用等，这有助于减少内存带宽需求和提高内存访问效率。

Triton高级抽象

Triton相比传统CUDA算子，通过更高层次的抽象显著简化了GPU编程，同时保持了高性能。

特性	CUDA	Triton
编程语言	C++扩展	Python原生
并行管理	手动划分线程块/线程	自动映射（program_id）
内存访问	显式优化合并访问/边界检查	智能加载/存储/掩码生成
计算原语	手动实现或调用库（如CUBLAS）	内置高阶算子（如tl.dot）支持操作融合
硬件适配	需针对不同架构手动调优	自动适配（如Tensor Core/TMA）

Triton简单算子开发

向量+向量示例——定义内核算子

```
@triton.jit
def add_kernel(x_ptr, y_ptr, output_ptr,n_elements,  BLOCK_SIZE: tl.constexpr, ):
    pid = tl.program_id(axis=0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    output = x + y
    tl.store(output_ptr + offsets, output, mask=mask)
```

https://triton-lang.org/main/getting-started/tutorials/index.html

Triton简单算子开发

向量加示例——调用内核算子

```
def add(x: torch.Tensor, y: torch.Tensor):
    output = torch.empty_like(x)
    assert x.is_cuda and y.is_cuda and output.is_cuda
    n_elements = output.numel()
    grid = lambda meta: (triton.cdiv(n_elements, meta['BLOCK_SIZE']), )
    add_kernel[grid](x, y, output, n_elements, BLOCK_SIZE=1024)
    return output
```

Triton简单算子开发

向量加示例——验证结果

```
torch.manual_seed(0)
size = 98432
x = torch.rand(size, device=DEVICE)
y = torch.rand(size, device=DEVICE)
output_torch = x + y
output_triton = add(x, y)
print(output_torch)
print(output_triton)
print(f'The maximum difference between torch and triton is '
      f'{torch.max(torch.abs(output_torch - output_triton))}')
```

Triton简单算子开发

运行命令：python 01-vector-add.py

```
(triton2.1.0) [duyb@Stream9 tutorials]$ python 01-vector-add.py
tensor([1.3713, 1.3076, 0.4940,  ..., 1.3374, 1.4960, 0.9115], device='cuda:0')
tensor([1.3713, 1.3076, 0.4940,  ..., 1.3374, 1.4960, 0.9115], device='cuda:0')
The maximum difference between torch and triton is 0.0
/public/home/duyb/anaconda3/envs/triton2.1.0/lib/python3.11/site-packages/triton/testing.py:304: UserWarning: FigureCanvasAgg is non-interactive, and thus
  plt.show()
vector-add-performance:
           size      Triton       Torch
0        4096.0   15.999999   15.999999
1        8192.0   24.000000   24.000000
2       16384.0   48.000000   48.000000
3       32768.0   90.352949   94.523077
4       65536.0  127.999995  127.999995
5      131072.0  192.000000  192.000000
6      262144.0  255.999991  250.137408
7      524288.0  279.272725  279.272725
8     1048576.0  307.200008  307.200008
9     2097152.0  319.168844  322.440352
10    4194304.0  327.714130  329.775424
11    8388608.0  332.108094  332.353736
12   16777216.0  334.367350  334.376254
13   33554432.0  335.508544  335.526441
14   67108864.0  335.938503  336.082067
15  134217728.0  336.225721  336.369531
```

Triton Puzzles Lite

Triton Puzzles 项目地址 https://github.com/srush/Triton-Puzzles

Triton Puzzles Lite 项目地址 https://github.com/SiriusNEO/Triton-Puzzles-Lite

```
#安装torch依赖
pip install torch==2.5.0
#克隆项目源码
git clone https://github.com/SiriusNEO/Triton-Puzzles-Lite.git
#检查项目示例是否可以通过测试
TRITON_INTERPRET=1 python3 puzzles_ans.py –a
#填写完成puzzles.py后运行，推荐使用Triton解释器模式，可以使用-p指定puzzle id
TRITON_INTERPRET=1 python3 puzzles.py –p 1
```

https://zhuanlan.zhihu.com/p/5964285807

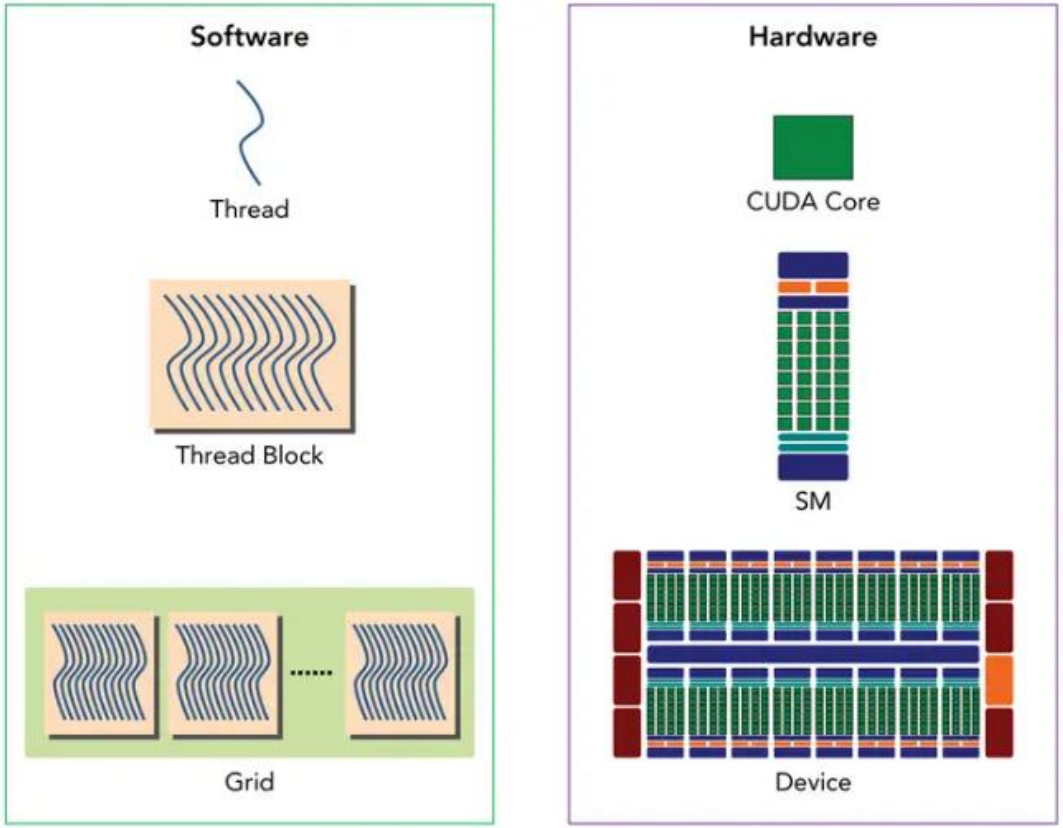
Demo 1

```python
@triton.jit
def demo1(x_ptr):
  range = tl.arange(0, 8)
  # print works in the interpreter
  print(range)
  x = tl.load(x_ptr + range, range < 5, 0)
  print(x)
def run_demo1():
  print("Demo1 Output: ")
  demo1[(1, 1, 1)](torch.ones(4, 3))
  print_end_line()
```

Demo 1

```python
@triton.jit
def demo1(x_ptr):
  range = tl.arange(0, 8)
  # print works in the interpreter
  print(range)
  x = tl.load(x_ptr + range, range < 5, 0)
  print(x)
def run_demo1():
  print("Demo1 Output: ")
  demo1[(1, 1, 1)](torch.ones(4, 3))
  print_end_line()
```

指定线程网格Grid的大小



Demo 1

```python
@triton.jit
def demo1(x_ptr):
  range = tl.arange(0, 8)
  # print works in the interpreter
  print(range)
  x = tl.load(x_ptr + range, range < 5, 0)
  print(x)
def run_demo1():
  print("Demo1 Output: ")
  demo1[(1, 1, 1)](torch.ones(4, 3))
  print_end_line()
```

创建一个4行3列的张量，每个元素都是1.0

```
[[1., 1., 1.],
 [1., 1., 1.],
 [1., 1., 1.],
 [1., 1., 1.]]
```

Demo 1

```python
@triton.jit
def demo1(x_ptr):
  range = tl.arange(0, 8)
  # print works in the interpreter
  print(range)
  x = tl.load(x_ptr + range, range < 5, 0)
  print(x)
def run_demo1():
  print("Demo1 Output: ")
  demo1[(1, 1, 1)](torch.ones(4, 3))
  print_end_line()
```

生成索引数组：range = [0 1 2 3 4 5 6 7]

Demo 1

```python
@triton.jit
def demo1(x_ptr):
  range = tl.arange(0, 8)
  # print works in the interpreter
  print(range)
  x = tl.load(x_ptr + range, range < 5, 0)
  print(x)
def run_demo1():
  print("Demo1 Output: ")
  demo1[(1, 1, 1)](torch.ones(4, 3))
  print_end_line()
```

生成索引数组：range = [0 1 2 3 4 5 6 7]

计算掩码：range < 5 = [1 1 1 1 1 0 0 0]

Demo 1

```python
@triton.jit
def demo1(x_ptr):
  range = tl.arange(0, 8)
  # print works in the interpreter
  print(range)
  x = tl.load(x_ptr + range, range < 5, 0)
  print(x)
def run_demo1():
  print("Demo1 Output: ")
  demo1[(1, 1, 1)](torch.ones(4, 3))
  print_end_line()
```

生成索引数组：range = [0 1 2 3 4 5 6 7]

计算掩码：range < 5 = [1 1 1 1 1 0 0 0]

load数据：x = [1. 1. 1. 1. 1. 0 0 0]

创建一个4行3列的张量，每个元素都是1.0

[[1., 1., 1.],
 [1., 1., 1.],
 [1., 1., 1.],
 [1., 1., 1.]]

Demo 2

```python
@triton.jit
def demo2(x_ptr):
    i_range = tl.arange(0, 8)[:, None]
    j_range = tl.arange(0, 4)[None, :]
    range = i_range * 4 + j_range
    # print works in the interpreter
    print(range)
    x = tl.load(x_ptr + range, (i_range < 4) & (j_range < 3), 0)
    print(x)
def run_demo2():
    print("Demo2 Output: ")
    demo2[(1, 1, 1)](torch.ones(4, 4))
    print_end_line()
```

Demo 2

```python
@triton.jit
def demo2(x_ptr):
    i_range = tl.arange(0, 8)[:, None]
    j_range = tl.arange(0, 4)[None, :]
    range = i_range * 4 + j_range
    # print works in the interpreter
    print(range)
    x = tl.load(x_ptr + range, (i_range < 4) & (j_range < 3), 0)
    print(x)
def run_demo2():
    print("Demo2 Output: ")
    demo2[(1, 1, 1)](torch.ones(4, 4))
    print_end_line()
```

创建一个4行4列的张量，每个元素都是1.0

```
[[1., 1., 1., 1.],
 [1., 1., 1., 1.],
 [1., 1., 1., 1.],
 [1., 1., 1., 1.]]
```

Demo 2

```python
@triton.jit
def demo2(x_ptr):
    i_range = tl.arange(0, 8)[:, None]
    j_range = tl.arange(0, 4)[None, :]
    range = i_range * 4 + j_range
    # print works in the interpreter
    print(range)
    x = tl.load(x_ptr + range, (i_range < 4) & (j_range < 3), 0)
    print(x)
def run_demo2():
    print("Demo2 Output: ")
    demo2[(1, 1, 1)](torch.ones(4, 4))
    print_end_line()
```

将一维索引向量变为8×1的列向量

```
[[0],
 [1],
 [2],
 [3],
 [4],
 [5],
 [6],
 [7]]
```

Demo 2

```python
@triton.jit
def demo2(x_ptr):
    i_range = tl.arange(0, 8)[:, None]
    j_range = tl.arange(0, 4)[None, :]
    range = i_range * 4 + j_range
    # print works in the interpreter
    print(range)
    x = tl.load(x_ptr + range, (i_range < 4) & (j_range < 3), 0)
    print(x)
def run_demo2():
    print("Demo2 Output: ")
    demo2[(1, 1, 1)](torch.ones(4, 4))
    print_end_line()
```

将一维索引向量变为1×4的行向量

[[0, 1, 2, 3]]

Demo 2

```python
@triton.jit
def demo2(x_ptr):
    i_range = tl.arange(0, 8)[:, None]
    j_range = tl.arange(0, 4)[None, :]
    range = i_range * 4 + j_range
    # print works in the interpreter
    print(range)
    x = tl.load(x_ptr + range,
      (i_range < 4) & (j_range < 3), 0)
    print(x)
def run_demo2():
    print("Demo2 Output: ")
    demo2[(1, 1, 1)](torch.ones(4, 4))
    print_end_line()
```

8×1的列向量和1×4的行向量会被广播扩展成8×4进行逐元素运算二维索引

```
[[0×4 + 0, 0×4 + 1, 0×4 + 2, 0×4 + 3],   # i = 0
 [1×4 + 0, 1×4 + 1, 1×4 + 2, 1×4 + 3],   # i = 1
 [2×4 + 0, 2×4 + 1, 2×4 + 2, 2×4 + 3],   # i = 2
 [3×4 + 0, 3×4 + 1, 3×4 + 2, 3×4 + 3],   # i = 3
 [4×4 + 0, 4×4 + 1, 4×4 + 2, 4×4 + 3],   # i = 4
 [5×4 + 0, 5×4 + 1, 5×4 + 2, 5×4 + 3],   # i = 5
 [6×4 + 0, 6×4 + 1, 6×4 + 2, 6×4 + 3],   # i = 6
 [7×4 + 0, 7×4 + 1, 7×4 + 2, 7×4 + 3]]   # i = 7
```

Demo 2

```python
@triton.jit
def demo2(x_ptr):
    i_range = tl.arange(0, 8)[:, None]
    j_range = tl.arange(0, 4)[None, :]
    range = i_range * 4 + j_range
    # print works in the interpreter
    print(range)
    x = tl.load(x_ptr + range, (i_range < 4) & (j_range < 3), 0)
    print(x)
def run_demo2():
    print("Demo2 Output: ")
    demo2[(1, 1, 1)](torch.ones(4, 4))
    print_end_line()
```

二维索引

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]
 [24 25 26 27]
 [28 29 30 31]]
```

掩码计算

```
[[ 1  1  1  0]
 [ 1  1  1  0]
 [ 1  1  1  0]
 [ 1  1  1  0]
 [ 0  0  0  0]
 [ 0  0  0  0]
 [ 0  0  0  0]
 [ 0  0  0  0]]
```

Demo 2

```python
@triton.jit
def demo2(x_ptr):
    i_range = tl.arange(0, 8)[:, None]
    j_range = tl.arange(0, 4)[None, :]
    range = i_range * 4 + j_range
    # print works in the interpreter
    print(range)
    x = tl.load(x_ptr + range, (i_range < 4) & (j_range < 3), 0)
    print(x)
def run_demo2():
    print("Demo2 Output: ")
    demo2[(1, 1, 1)](torch.ones(4, 4))
    print_end_line()
```

二维索引

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]
 [24 25 26 27]
 [28 29 30 31]]
```

掩码计算

```
[[ 1  1  1  0]
 [ 1  1  1  0]
 [ 1  1  1  0]
 [ 1  1  1  0]
 [ 0  0  0  0]
 [ 0  0  0  0]
 [ 0  0  0  0]
 [ 0  0  0  0]]
```

```
[[1., 1., 1., 1.],
 [1., 1., 1., 1.],
 [1., 1., 1., 1.],
 [1., 1., 1., 1.]]
```

load数据：x =

```
[[1. 1. 1. 0.]
 [1. 1. 1. 0.]
 [1. 1. 1. 0.]
 [1. 1. 1. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

Demo 3

```
tl.store(ptr, value, mask)
```

```
@triton.jit
def demo3(z_ptr):
    range = tl.arange(0, 8)
    z = tl.store(z_ptr + range, 10, range < 5)
def run_demo3():
    print("Demo3 Output: ")
    z = torch.ones(4, 3)
    demo3[(1, 1, 1)](z)
    print(z)
    print_end_line()
```

Demo 3

```
@triton.jit
def demo3(z_ptr):
    range = tl.arange(0, 8)
    z = tl.store(z_ptr + range, 10, range < 5)
def run_demo3():
    print("Demo3 Output: ")
    z = torch.ones(4, 3)
    demo3[(1, 1, 1)](z)
    print(z)
    print_end_line()
```

创建一个4行3列的张量，每个元素都是1.0

```
[[1., 1., 1.],
 [1., 1., 1.],
 [1., 1., 1.],
 [1., 1., 1.]]
```

Demo 3

```
@triton.jit
def demo3(z_ptr):
    range = tl.arange(0, 8)
    z = tl.store(z_ptr + range, 10, range < 5)
def run_demo3():
    print("Demo3 Output: ")
    z = torch.ones(4, 3)
    demo3[(1, 1, 1)](z)
    print(z)
    print_end_line()
```

```
[[1., 1., 1.],
 [1., 1., 1.],
 [1., 1., 1.],
 [1., 1., 1.]]
```

→

```
[[10., 10., 10.],
 [10., 10., 1.],
 [1., 1., 1.],
 [1., 1., 1.]]
```

Demo 4

```python
@triton.jit
def demo4(x_ptr):
    pid = tl.program_id(0)
    range = tl.arange(0, 8) + pid * 8
    x = tl.load(x_ptr + range, range < 20)
    print("Print for each", pid, x)
def run_demo4():
    print("Demo4 Output: ")
    x = torch.ones(2, 4, 4)
    demo4[(3, 1, 1)](x)
    print_end_line()
```

grid配置为(3, 1, 1)
表示沿着第0个轴有3个并行block，
其他两个轴（axis=1和axis=2）各有1个block

Demo 4

```
@triton.jit
def demo4(x_ptr):
    pid = tl.program_id(0)
    range = tl.arange(0, 8) + pid * 8
    x = tl.load(x_ptr + range, range < 20)
    print("Print for each", pid, x)
def run_demo4():
    print("Demo4 Output: ")
    x = torch.ones(2, 4, 4)
    demo4[(3, 1, 1)](x)
    print_end_line()
```

每个块对应有一个program id

用于沿着第0个轴(axis=0)获取当前block的program id

沿着第0个轴有3个并行block，对应pid=0, 1, 2

Demo 4

```
@triton.jit
def demo4(x_ptr):
    pid = tl.program_id(0)
    range = tl.arange(0, 8) + pid * 8
    x = tl.load(x_ptr + range, range < 20)
    print("Print for each", pid, x)
def run_demo4():
    print("Demo4 Output: ")
    x = torch.ones(2, 4, 4)
    demo4[(3, 1, 1)](x)
    print_end_line()
```

tl.arange(0, 8)生成了从0到7的索引序列
表示每个block处理8个元素

沿着第0个轴有3个并行block，对应pid=0, 1, 2

Demo 4

pid=0： range = [0, 1, 2, 3, 4, 5, 6, 7]
pid=1： range = [8, 9, 10, 11, 12, 13, 14, 15]
pid=2： range = [16, 17, 18, 19, 20, 21, 22, 23]

```
@triton.jit
def demo4(x_ptr):
    pid = tl.program_id(0)
    range = tl.arange(0, 8) + pid * 8
    x = tl.load(x_ptr + range, range < 20)
    print("Print for each", pid, x)
def run_demo4():
    print("Demo4 Output: ")
    x = torch.ones(2, 4, 4)
    demo4[(3, 1, 1)](x)
    print_end_line()
```

pid *8用于根据当前block的pid计算起始位置
两者相加表示当前block要处理的索引范围

沿着第0个轴有3个并行block，对应pid=0, 1, 2

Demo 4

pid=0： range = [0, 1, 2, 3, 4, 5, 6, 7]
pid=1： range = [8, 9, 10, 11, 12, 13, 14, 15]
pid=2： range = [16, 17, 18, 19, 20, 21, 22, 23]

```python
@triton.jit
def demo4(x_ptr):
    pid = tl.program_id(0)
    range = tl.arange(0, 8) + pid * 8
    x = tl.load(x_ptr + range, range < 20)
    print("Print for each", pid, x)
def run_demo4():
    print("Demo4 Output: ")
    x = torch.ones(2, 4, 4)
    demo4[(3, 1, 1)](x)
    print_end_line()
```

掩码为range<20
对于pid = 2，只加载[16, 17, 18, 19]
其余部分被忽略

Demo 4

pid=0： range = [0, 1, 2, 3, 4, 5, 6, 7]
pid=1： range = [8, 9, 10, 11, 12, 13, 14, 15]
pid=2： range = [16, 17, 18, 19, 20, 21, 22, 23]

```python
@triton.jit
def demo4(x_ptr):
    pid = tl.program_id(0)
    range = tl.arange(0, 8) + pid * 8
    x = tl.load(x_ptr + range, range < 20)
    print("Print for each", pid, x)
def run_demo4():
    print("Demo4 Output: ")
    x = torch.ones(2, 4, 4)
    demo4[(3, 1, 1)](x)
    print_end_line()
```

三维张量
第一维有2个(4, 4)
每个元素都是1

```
[[[1., 1., 1., 1.],
  [1., 1., 1., 1.],
  [1., 1., 1., 1.],
  [1., 1., 1., 1.]],
 [[1., 1., 1., 1.],
  [1., 1., 1., 1.],
  [1., 1., 1., 1.],
  [1., 1., 1., 1.]]]
```

Demo 4

pid=0: range = [0, 1, 2, 3, 4, 5, 6, 7]
pid=1: range = [8, 9, 10, 11, 12, 13, 14, 15]
pid=2: range = [16, 17, 18, 19, 20, 21, 22, 23]

```
@triton.jit
def demo4(x_ptr):
    pid = tl.program_id(0)
    range = tl.arange(0, 8) + pid * 8
    x = tl.load(x_ptr + range, range < 20)
    print("Print for each", pid, x)
def run_demo4():
    print("Demo4 Output: ")
    x = torch.ones(2, 4, 4)
    demo4[(3, 1, 1)](x)
    print_end_line()
```

Print for each [0] [1. 1. 1. 1. 1. 1. 1. 1.]
Print for each [1] [1. 1. 1. 1. 1. 1. 1. 1.]
Print for each [2] [1. 1. 1. 1. 0. 0. 0. 0.]

```
[[[1., 1., 1., 1.],
  [1., 1., 1., 1.],
  [1., 1., 1., 1.],
  [1., 1., 1., 1.]],
 [[1., 1., 1., 1.],
  [1., 1., 1., 1.],
  [1., 1., 1., 1.],
  [1., 1., 1., 1.]]]
```

Puzzle 1：Constant Add

编写要求：向量和常数相加，使用一个program id axis，块大小B0总是和向量x的长度N0相等

计算公式： $z_i = 10 + x_i \quad \text{for}\, i = 1 \ldots N_0$

python 实现：

```
def add_spec(x: Float32[32,]) -> Float32[32,]:
    "This is the spec that you should implement. Uses typing to define sizes."
    return x + 10.0
```

Puzzle 1：Constant Add

编写要求：向量和常数相加，使用一个program id axis，块大小B0总是和向量x的长度N0相等

计算公式： $z_i = 10 + x_i \quad \text{for}\, i = 1 \ldots N_0$

triton 实现：

```
@triton.jit
def add_kernel(x_ptr, z_ptr, N0, B0: tl.constexpr):
    # We name the offsets of the pointers as "off_"
    off_x = tl.arange(0, B0)
    x = tl.load(x_ptr + off_x)
    # Finish me!
    x = x + 10.0
    tl.store(z_ptr + off_x, x)
    return
```

Puzzle 2：Constant Add Block

编写要求：向量和常数相加，使用一个program id axis（无for循环），块大小B0现在小于向量x的长度N0

计算公式： $z_i = 10 + x_i \quad \text{for}\, i = 1 \ldots N_0$

python 实现：

```
def add_spec(x: Float32[32,]) -> Float32[32,]:
    return x + 10.0
```

Puzzle 2：Constant Add Block

编写要求：向量和常数相加，使用一个program id axis（无for循环），块大小B0现在小于向量x的长度N0

计算公式： $z_i = 10 + x_i \quad \text{for}\, i = 1 \ldots N_0$

triton 实现：

```
@triton.jit
def add_mask2_kernel(x_ptr, z_ptr, N0, B0: tl.constexpr):
    # Finish me!
    return
```

Puzzle 2：Constant Add Block

编写要求：向量和常数相加，使用一个program id axis（无for循环），块大小B0现在小于向量x的长度N0：

```
@triton.jit
def add_mask2_kernel(x_ptr, z_ptr, N0, B0: tl.constexpr):
    # Finish me!
    pid = tl.program_id(0)
    off_x = pid * B0 + tl.arange(0, B0)
    mask = off_x < N0
    x = tl.load(x_ptr + off_x, mask)
    x = x + 10.0
    tl.store(z_ptr + off_x, x, mask)
    return
```

Puzzle 3：Outer Vector Add

编写要求：两个向量相加，使用一个program id axis，块大小B0始终与向量x长度N0相同，块大小B1始终与向量y长度N1相同。

计算公式： $z_{j,i} = x_i + y_i \quad \text{for } i = 1 \ldots B_0, j = 1 \ldots B_1$

python 实现：

```
def add_vec_spec(x: Float32[32,], y: Float32[32,]) -> Float32[32, 32]:
    return x[None, :] + y[:, None]
```

假设输入向量 x 和 y：

x = [1.0, 2.0, 3.0] # N0 = 3

y = [4.0, 5.0] # N1 = 2

x [None, :] = [[1.0 , 2.0, 3.0]]

y [:, None] = [[4.0],

[5,0]]

Puzzle 3：Outer Vector Add

编写要求：两个向量相加，使用一个program id axis，块大小B0始终与向量x长度N0相同，块大小B1始终与向量y长度N1相同。

计算公式： $z_{j,i} = x_i + y_i \quad \text{for } i = 1 \ldots B_0, j = 1 \ldots B_1$

python 实现：

```
def add_vec_spec(x: Float32[32,], y: Float32[32,]) -> Float32[32, 32]:
    return x[None, :] + y[:, None]
```

x [None, :]和 y[:, None]相加需要广播后逐个元素相加

x [None, :] = [[1.0 , 2.0, 3.0]]

y [:, None] = [[4.0],

[5,0]]

$$z = \begin{bmatrix} 1.0+4.0 & 2.0+4.0 & 3.0+4.0 \\ 1.0+5.0 & 2.0+5.0 & 3.0+5.0 \end{bmatrix} = \begin{bmatrix} 5.0 & 6.0 & 7.0 \\ 6.0 & 7.0 & 8.0 \end{bmatrix}$$

Puzzle 3：Outer Vector Add

编写要求：两个向量相加，使用一个program id axis，块大小B0始终与向量x长度N0相同，块大小B1始终与向量y长度N1相同。

$$z = \begin{bmatrix} 5.0 & 6.0 & 7.0 \\ 6.0 & 7.0 & 8.0 \end{bmatrix}$$

```python
@triton.jit
def add_vec_kernel(x_ptr, y_ptr, z_ptr, N0, N1, B0: tl.constexpr, B1: tl.constexpr):
    # Finish me!
    pid = tl.program_id(0)
    off_x = pid * B0 + tl.arange(0, B0)
    off_y = pid * B1 + tl.arange(0, B1)
    off_z = off_y[:, None] * B0 + off_x[None, :]
    x = tl.load(x_ptr + off_x)
    y = tl.load(y_ptr + off_y)
    z = x[None, :] + y[:, None]
    tl.store(z_ptr + off_z, z)
    return
```

off_x = [0, 1, 2]

off_y = [0, 1]

$$\text{off_z} = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \end{bmatrix}$$

$$\text{z_off} = \begin{bmatrix} 0+0 & 0+1 & 0+2 \\ 1*3+0 & 1*3+1 & 1*3+2 \end{bmatrix}$$

Puzzle 4：Outer Add Block

编写要求：行向量和列向量相加，使用2个program id axis，块大小B0始终小于向量x长度N0，块大小B1始终小于向量y长度N1。

计算公式： $z_{j,i} = x_i + y_i \quad \text{for } i = 1 \ldots N_0, j = 1 \ldots N_1$

python 实现：

```
def add_vec_block_spec(x: Float32[100,], y: Float32[90,]) -> Float32[90, 100]:
    return x[None, :] + y[:, None]
```

Puzzle 4：Outer Add Block

编写要求：行向量和列向量相加，使用2个program id axis，块大小B0始终小于向量x长度N0，块大小B1始终小于向量y长度N1。

```
@triton.jit
def add_vec_kernel(x_ptr, y_ptr, z_ptr, N0, N1, B0: tl.constexpr, B1: tl.constexpr):
    block_id_x = tl.program_id(0)
    block_id_y = tl.program_id(1)
    # Finish me!
    off_x = block_id_x * B0 + tl.arange(0, B0)
    off_y = block_id_y * B1 + tl.arange(0, B1)
    off_z = off_y[:, None] * N0 + off_x[None, :]
    mask_x = off_x < N0
    mask_y = off_y < N1
    mask_z =
    x = tl.load(x_ptr + off_x, mask_x)
    y = tl.load(y_ptr + off_y, mask_y)
    z = x[None, :] + y[:, None]
    tl.store(z_ptr + off_z, z, mask_z)
    return
```

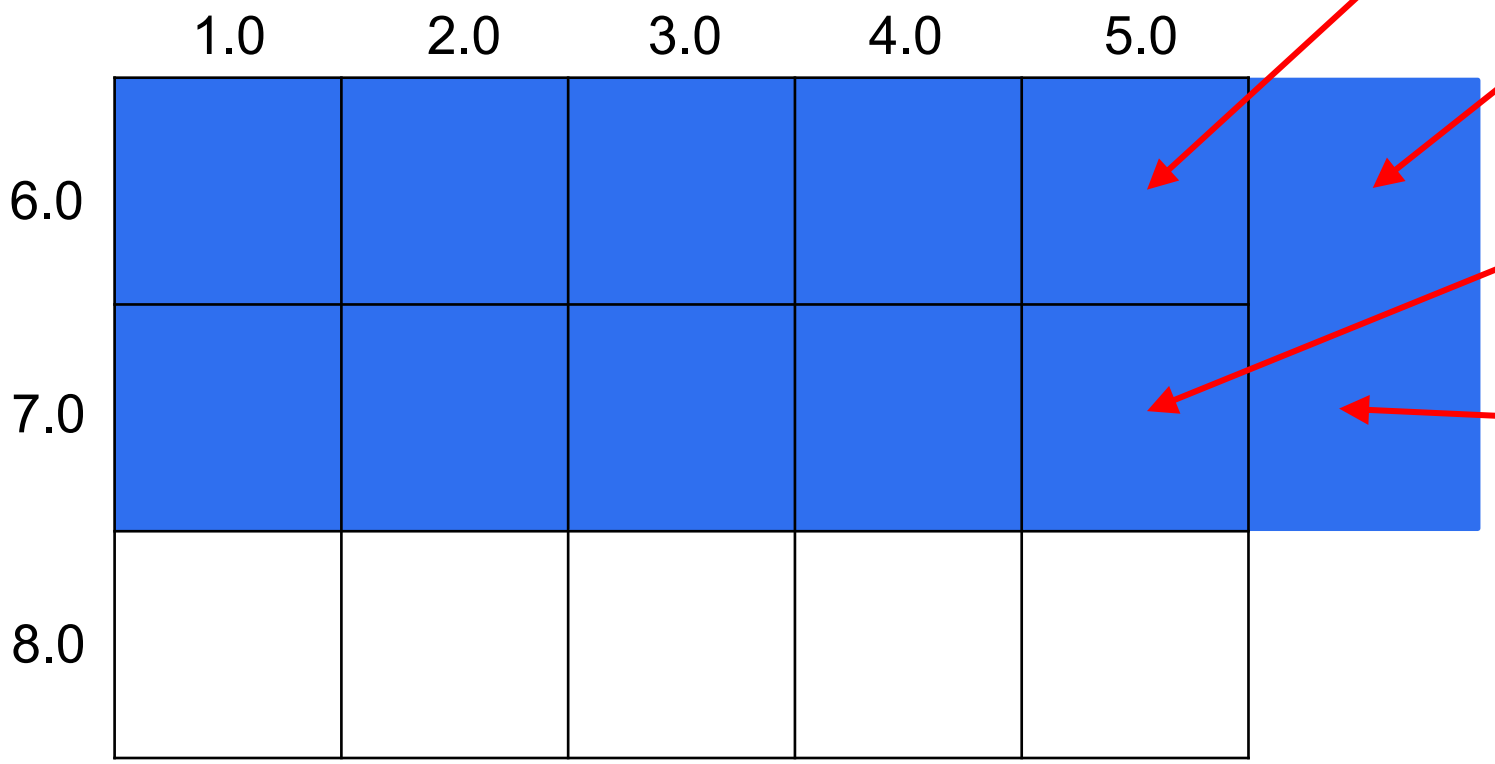
Puzzle 4：Outer Add Block

```
off_x = block_id_x * B0 + tl.arange(0, B0)
off_y = block_id_y * B1 + tl.arange(0, B1)
```

假设输入向量 x 和 y，分块大小 B0 = B1 = 2：

x = [1.0, 2.0, 3.0, 4.0, 5.0] # N0 = 5

y = [6.0, 7.0, 8.0] # N1 = 3



off_x = 2*2+0 = 4, mask_x = 1
off_y = 0*2+0 = 0, mask_y = 1
mask_z = 1

off_x = 2*2+1 = 5, mask_x = 0
off_y = 0*2+0 = 0, mask_y = 1
mask_z = 0

off_x = 2*2+0 = 4, mask_x = 1
off_y = 0*2+1 = 1, mask_y = 1
mask_z = 1

off_x = 2*2+1 = 5, mask_x = 0
off_y = 0*2+1 = 1, mask_y = 1
mask_z = 0

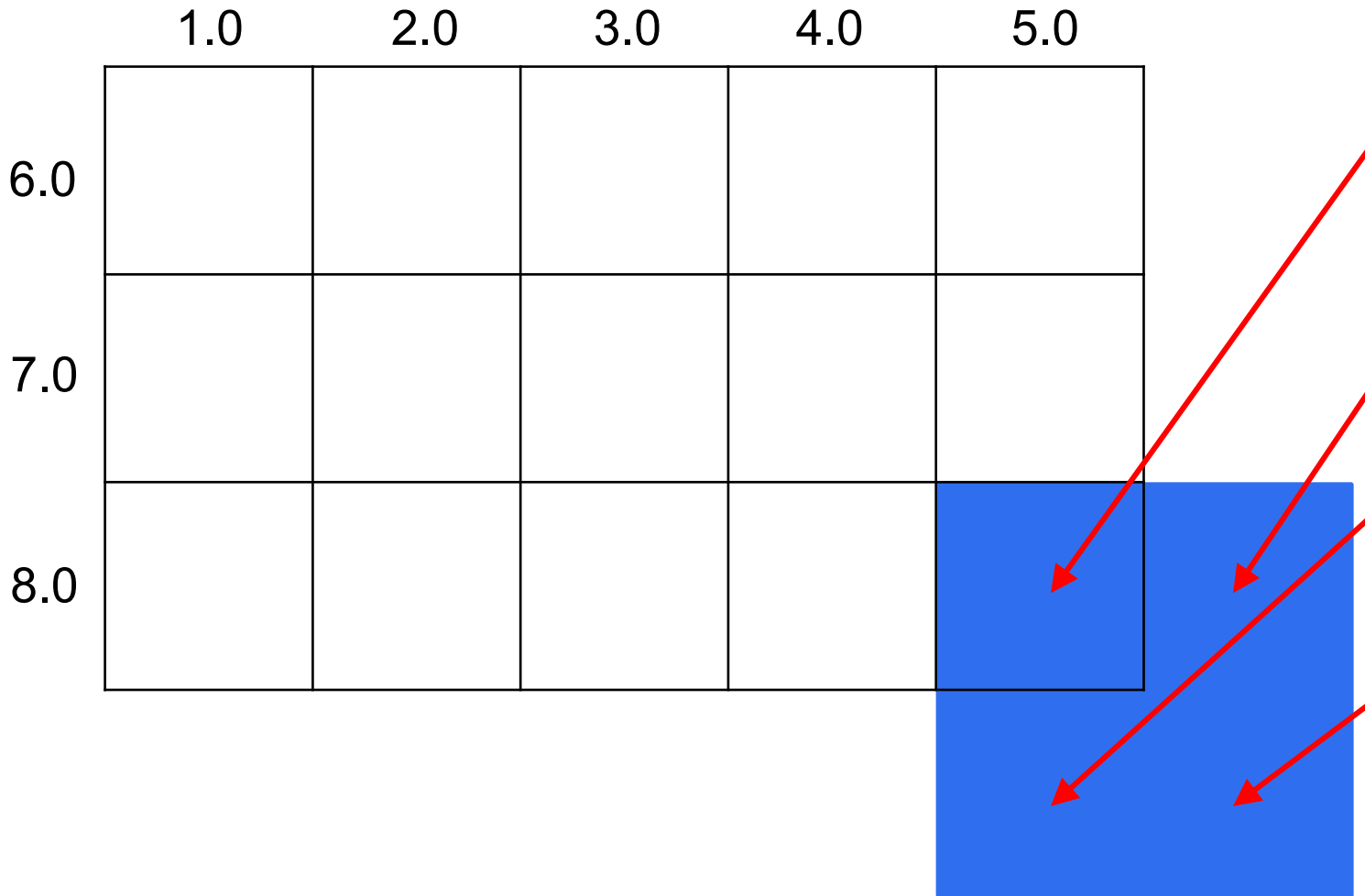
Puzzle 4：Outer Add Block

```
off_x = block_id_x * B0 + tl.arange(0, B0)
off_y = block_id_y * B1 + tl.arange(0, B1)
```

假设输入向量 x 和 y，分块大小 B0 = B1 = 2：

x = [1.0, 2.0, 3.0, 4.0, 5.0] # N0 = 5

y = [6.0, 7.0, 8.0] # N1 = 3



off_x = 2*2+0 = 4, mask_x = 1
off_y = 1*2+0 = 0, mask_y = 1
mask_z = 1

off_x = 2*2+1 = 5, mask_x = 0
off_y = 1*2+0 = 2, mask_y = 1
mask_z = 0

off_x = 2*2+0 = 4, mask_x = 1
off_y = 1*2+1 = 3, mask_y = 0
mask_z = 0

off_x = 2*2+1 = 5, mask_x = 0
off_y = 1*2+1 = 3, mask_y = 0
mask_z = 0

Puzzle 4：Outer Add Block

编写要求：行向量和列向量相加，使用2个program id axis，块大小B0始终小于向量x长度N0，块大小B1始终小于向量y长度N1。

```
@triton.jit
def add_vec_kernel(x_ptr, y_ptr, z_ptr, N0, N1, B0: tl.constexpr, B1: tl.constexpr):
    block_id_x = tl.program_id(0)
    block_id_y = tl.program_id(1)
    # Finish me!
    off_x = block_id_x * B0 + tl.arange(0, B0)
    off_y = block_id_y * B1 + tl.arange(0, B1)
    off_z = off_y[:, None] * N0 + off_x[None, :]
    mask_x = off_x < N0
    mask_y = off_y < N1
    mask_z = mask_x[None, :] & mask_y[:, Mone]
    x = tl.load(x_ptr + off_x, mask_x)
    y = tl.load(y_ptr + off_y, mask_y)
    z = x[None, :] + y[:, None]
    tl.store(z_ptr + off_z, z, mask_z)
    return
```

Puzzle 5：Fused Outer Mutiplication

编写要求：行向量和列向量相乘后进行relu操作，使用2个program id axis，块大小B0始终小于向量x长度N0，块大小B1始终小于向量y长度N1。

计算公式：$z_{j,i} = \mathrm{relu}(x_i \times \mathrm{y}_i) \quad \mathrm{for}\, i = 1 \ldots N_0, j = 1 \ldots N_1$

python 实现：

```
def mul_relu_block_spec(x: Float32[100,], y: Float32[90,]) -> Float32[90, 100]:
    return torch.relu(x[None, :] * y[:, None])
```

$$\mathrm{relu}(x) = \max(0, x)$$

Puzzle 5：Fused Outer Mutiplication

编写要求：行向量和列向量相乘后进行relu操作，使用2个program id axis，块大小B0始终小于向量x长度N0，块大小B1始终小于向量y长度N1。

```
@triton.jit
def add_vec_kernel(x_ptr, y_ptr, z_ptr, N0, N1, B0: tl.constexpr, B1: tl.constexpr):
    block_id_x = tl.program_id(0)
    block_id_y = tl.program_id(1)
    # Finish me!
    off_x = block_id_x * B0 + tl.arange(0, B0)
    off_y = block_id_y * B1 + tl.arange(0, B1)
    off_z = off_y[:, None] * N0 + off_x[None, :]
    mask_x = off_x < N0
    mask_y = off_y < N1
    mask_z = mask_x[None, :] & mask_y[:, Mone]
    x = tl.load(x_ptr + off_x, mask_x)
    y = tl.load(y_ptr + off_y, mask_y)
    z = x[None, :] * y[:, None]
    z_relu = tl.where(z>0, z, 0)
    tl.store(z_ptr + off_z, relu_z, mask_z)
    return
```

关键参数优化

```
@triton.jit
def matmul_kernel(
        a_ptr, b_ptr, c_ptr,M, N, K,
        stride_am, stride_ak, stride_bk, stride_bn, stride_cm, stride_cn,
        BLOCK_SIZE_M: tl.constexpr, BLOCK_SIZE_N: tl.constexpr, BLOCK_SIZE_K: tl.constexpr,
        GROUP_SIZE_M: tl.constexpr,
        ):
```

```
def call(args):
    with torch.cuda._DeviceGuard(0):
        torch.cuda.set_device(0)
        grid = lambda META: (triton.cdiv(512, META['BLOCK_SIZE_M']) * triton.cdiv(512, META['BLOCK_SIZE_N']), )
        matmul_kernel[grid](*args, BLOCK_SIZE_M=32, BLOCK_SIZE_N=32, BLOCK_SIZE_K=32, GROUP_SIZE_M=8,num_stages=8,num_warps=4)
```

```
if __name__ == '__main__':
  from torch._inductor.runtime.benchmarking import benchmarker
  args = get_args()
  ms = benchmarker.benchmark_gpu(lambda: call(args), rep=40, fast_flush=True)
  num_gb = 0.0016
  gb_per_s = num_gb / (ms / 1e3)
  print(f"{ms:.3f}ms    {num_gb:.3f}GB    {gb_per_s:.2f}GB/s")
```

关键参数优化

参数调整需要综合考虑GPU的硬件限制和算子任务特性：

- 根据GPU最大线程数调整num_warps
- 优化BLOCK_SIZE以平衡计算量和资源使用
  - 考虑共享内存的使用
  - 考虑寄存器的使用
- 根据计算规模调整动态调整参数

Triton调优方式

- 运行时传参

  用户在程序运行时传递自定义配置参数

- @triton.autotune

  使用自动调优方法从给定Config列表中选择kernel的最优配置参数

- @triton.heuristics

  使用启发式算法计算Kernel配置参数

@triton.autotune

@triton.autotune允许开发者为Kernel函数中的BLOCK_SIZE、num_warps等关键参数设定候选值或范围，每一组参数配置都会编译生成可执行文件，并通过测试选取执行时间最短的配置作为最佳参数配置。Kernel调优中配置的Config对象包括以下参数：

- kwargs：用于给内核传递关键字参数，例如可以设置BLOCK_SIZE用于确定块大小
- num_warps：设置编译时内核使用的线程数
- num_stages：设置共享内存缓存区数量，多用于矩阵乘法调整流水线循环的阶段数
- num_ctas：设置块集群的块数，仅适用于SM90+
- maxnreg：设置单个线程可以使用的最大寄存器数

@triton.autotune

@triton.autotune(configs, key, prune_configs_by=None, reset_to_zero=None, restore_value=None, pre_hook=None, post_hook=None, warmup=25, rep=100, use_cuda_graph=False)

- configs：triton.Config对象列表
- key：参数名列表，当值发生改变时将触发对所有配置的解析。
- prune_configs_by：对配置剪枝的函数字典
- reset_to_zore：每次配置解析之前重置指定参数为0
- restore_value：每次配置解析之后恢复指定参数值
- pre_hook：设置调用Kernel之前被调用的函数
- post_hook：设置调用Kernel之后被调用的函数
- warmup：指定基准测试的预热时间
- rep：指定基准测试的重复时间
- use_cuda_graph：是否使用CUDA Graph

@triton.autotune

1、在@triton.jit前添加@triton.autotune

```
@triton.autotune(
    configs=[
        triton.Config({'BLOCK_SIZE_M': 128, 'BLOCK_SIZE_N': 64, 'BLOCK_SIZE_K': 32, 'GROUP_SIZE_M': 8}, num_stages=4,num_warps=4),
        triton.Config({'BLOCK_SIZE_M': 64, 'BLOCK_SIZE_N': 128, 'BLOCK_SIZE_K': 32, 'GROUP_SIZE_M': 8}, num_stages=4,num_warps=4),
        triton.Config({'BLOCK_SIZE_M': 128, 'BLOCK_SIZE_N': 32, 'BLOCK_SIZE_K': 32, 'GROUP_SIZE_M': 8}, num_stages=4,num_warps=4),
        triton.Config({'BLOCK_SIZE_M': 128, 'BLOCK_SIZE_N': 32, 'BLOCK_SIZE_K': 64, 'GROUP_SIZE_M': 8}, num_stages=4,num_warps=4)
    ],
    key=['M', 'N', 'K'],)
@triton.jit
def matmul_kernel(
            ……
```

2、kernel调用时无需再指定参数

```
matmul_kernel[grid](*args, BLOCK_SIZE_M=128, BLOCK_SIZE_N=128, BLOCK_SIZE_K=32, GROUP_SIZE_M=32,num_stages=2,num_warps=4)
```

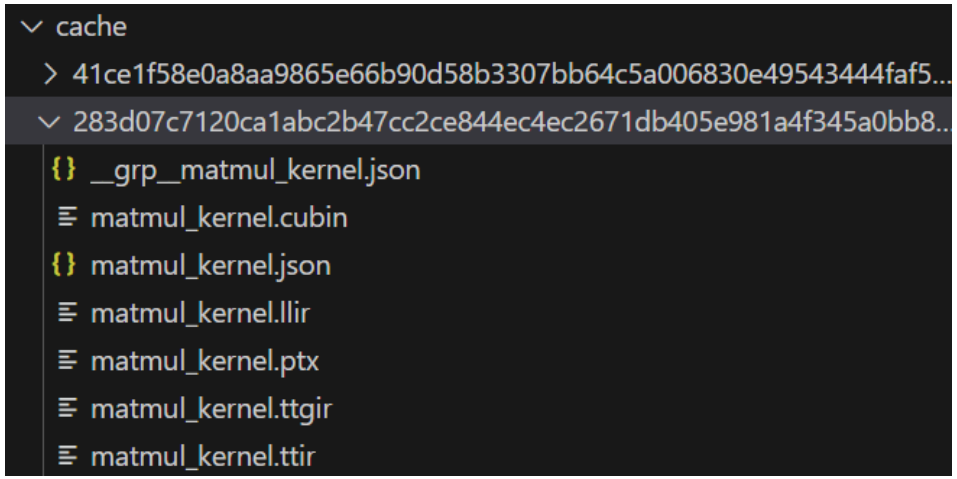
```
matmul_kernel[grid](*args)
```

@triton.autotune

通过添加调试选项*TRITON_PRINT_AUTOTUNING*，可打印自动调优所花费的时间和最佳配置

```
(torch2.5.0) [duyb@Stream9 lc]$ export TRITON_PRINT_AUTOTUNING=1
(torch2.5.0) [duyb@Stream9 lc]$ python matmul.py
Triton autotuning for function matmul_kernel finished after 0.70s; best config selected: BLOCK_SIZE_M: 128, BLOCK_SIZE_N: 32,
BLOCK_SIZE_K: 32, GROUP_SIZE_M: 8, num_warps: 4, num_ctas: 1, num_stages: 4, maxnreg: None;
0.019ms    0.002GB    85.62GB/s
```

.triton/cache中每条Config都会生成编译后的可执行文件



@triton.heuristics

@triton.heuristics允许在运行时根据输入数据、硬件架构等条件调整计算配置，而不是使用固定的参数，可以用于动态选择最优的计算参数。该装饰器接受一个values参数，它是一字典：

- 键（key）是元参数名称。
- 值（value）是一个 函数 ，用于根据输入参数计算该元参数的值，这个函数的输入是一个 位置参数列表（list[Any]），可以基于这些参数计算所需的元参数。

@triton.heuristics

在自动调优成本过高或不适用时，使用@triton.heuristics可以指定参数的计算方式。在定义好计算元参数值的函数后，使用方式和@triton.autotune一致

```
@triton.heuristics(
 values={
 'BLOCK_SIZE_M': lambda args: triton.next_power_of_2(args['M']) // 4,
 'BLOCK_SIZE_N': lambda args: triton.next_power_of_2(args['N']) // 4,
 'BLOCK_SIZE_K': lambda args: 32,
 'GROUP_SIZE_M': lambda args: 8 ,
})
#(M,K)*(K,N)=(M,N)  其中M、N、K为两个矩阵维度的shape值
#M、N的blocksize值一般为小于shape值的最大二次幂再除以4为较优
#shape较小时,K的blocksize的取值范围一般[32,64,128,256]为最优
@triton.jit
def matmul_kernel(
  ……
```

Inductor调优方式

Inductor的heuristics是TorchInductor针对自动生成的Triton Kernel进行自动调优的启发式方法，针对生成的pointwise、reduction和persisitent_reduction三类Triton Kernel分别设计了启发式调优方法，下面首先介绍这三类kernel的特征：

- pointwise：无归约计算，Kernel名称为triton_poi_xxx
- reduction：带for循环的归约计算，Kernel名称为triton_red_xxx
- persistent_reduction：不带for循环的归约计算，Kernel名称体现为triton_per_xxx

Inductor调优方式

```
@triton_heuristics.persistent_reduction(
    size_hints=[16384, 1024],
    reduction_hint=ReductionHint.INNER,
    filename=__file__,
    triton_meta={'signature': {0: '*i64', 1: '*fp16', 2: '*fp16', 3: '*i64', 4: '*fp16', 5: '*fp16', 6: '*fp16', 7: '*fp16', 8:
'*fp16', 9: 'i32', 10: 'i32'}, 'device': DeviceProperties(type='cuda', index=0, cc=80, major=8, regs_per_multiprocessor=65536,
max_threads_per_multi_processor=2048, multi_processor_count=108), 'constants': {}, 'configs': [AttrsDescriptor(divisible_by_16=
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10), equal_to_1=())]},
    inductor_meta={'autotune_hints': set(), 'kernel_name': 'triton_per_fused_add_div_embedding_mean_mul_std_sub_1',
'mutated_arg_names': [], 'no_x_dim': True, 'num_load': 5, 'num_reduction': 4, 'backend_hash':
'6DEFDAD8522DFEBDF4A8881EB1D14DD39B8A3B932BF9FB2371EDAA1CB2F14DC7', 'are_deterministic_algorithms_enabled': False,
'assert_indirect_indexing': True, 'autotune_local_cache': True, 'autotune_pointwise': True, 'autotune_remote_cache': None,
'force_disable_caches': False, 'dynamic_scale_rblock': True, 'max_autotune': False, 'max_autotune_pointwise': False,
'min_split_scan_rblock': 256, 'spill_threshold': 16, 'store_cubin': False, 'kernel_num_gb': 0.076553728}
)
@triton.jit
def triton_per_fused_add_div_embedding_mean_mul_std_sub_1(in_ptr0, in_ptr1, in_ptr2, in_ptr3, in_ptr4, in_ptr5, in_ptr6,
out_ptr0, out_ptr3, xnumel, rnumel):
    ……
```

```
    x2 = (xindex // 16384)
    x3 = xindex
    tmp0 = tl.load(in_ptr0 + (x0 + (128*x2)), None, eviction_policy='evict_last')
    tmp1 = tl.full([1], 0, tl.int64)
    tmp2 = tmp0 > tmp1
    tl.store(out_ptr0 + (x3), tmp2, None)
```

pointwise启发式调优

1、根据size_hints中的元素数量numel计算块大小block_size（256至1024之间）

2、根据inductor_meta中传入的autotune_hints自动调优提示生成配置，目前仅有ONE_ELEMENT_PER_THREAD每个线程只处理一个元素的提示，会根据不同维度生成不同维度上线程处理元素的数量：

维数	BLOCK配置
一维	(block_size//4)
二维	(block_size//4, 1)(1, block_size//4)
三维	(block_size//4, 1, 1)(1, block_size//4, 1)(1, 1, block_size//4)

3、如果禁用自动调优或者没有启用max-autotune，则会使用不同维度下的默认配置，否则会根据不同维度生成相应的其他配置

维数	默认配置	其他配置
一维	(block_size)	(block_size//2), num_elemnets_per_warp=64
二维	(32,32)	(32,32);(64,64);(256,16);(16,256);(block_size,1);(1,block_size)
三维	(16,16,16)	(16,16,16);(64,8,8);(8,64,8);(8,8,64);(block_size,1,1);(1,block_size,1);(1,1,block_size)

4、这些配置会传递给cached_autotune函数，该函数会检查缓存，可以在调优时减少重复调优进而提高效率。

https://github.com/pytorch/pytorch/blob/main/torch/_inductor/runtime/triton_heuristics.py

reduction启发式调优

1、根据 size_hints (计算乘积归约为一维)和 inductor_meta确定是否需要调整 RBLOCK，如果 size_hints[x] >= 1024 且load和reduction指令总数较高(>=10)时，认为操作可能需要更多的寄存器，从而将 RBLOCK 设置为较小的值(不超过MAX_R0_BLOCK=1024)，否则MAX_R0_BLOCK默认为2048。

2、根据RedutionHint提示或者Inductor_meta传入的信息去初始配置

ReductionHint	BLOCK配置
INNER	(1,rnumel if 256<=rnumel<MAX_R0_BLOCK else MAX_R0_BLOCK)
OUTER	(64,8)
OUTER_TINY	(2*(256//rnumel) if rnumel<=256 else 1, min(rnumel, MAX_R0_BLOCK))
禁用pointwise_autotuning	(32,128)
其他配置	(64,64);(8,512);(64,4,num_warps=8)

3、将初始配置传入triton_config_reduction函数对配置进行调整，核心逻辑是在GPU计算限制范围内，动态调整归约块大小，首先扩展块大小以充分利用计算资源，再收缩块大小确保不会超出GPU限制，同时动态调整warp数量，提高并行度并减少寄存器占用，最终生成配置。

- 初始化归约块大小，计算rnumels，并限制x不超过size_hints[x]
- 确定目标计算规模，计算total_numel作为参考值，必要时缩小target(最大减少8倍)
- 扩展块大小。逐步扩展x维度和rnumels，使计算规模接近target
- 计算warp数量，根据total_numel确定num_warps，并根据是否是寄存器密集型进行调整(persistent_reduction是计算密集型，需要减半max_num_warps上限)
- 调整x维度以符合网格限制，确保x不超过最大网格大小(满足num_blocks * num_warps * warp_size < max_grid_x)
- 优化归约块大小，如果total_numel仍然过大则缩小rnumels

https://github.com/pytorch/pytorch/blob/main/torch/_inductor/runtime/triton_heuristics.py

persistent_reduction启发式调优

1、遍历给定的xblock参数作为候选配置(xblock=1,8,32,128，其中1始终保留，其他需要满足rnumel * xblock≤4096且xblock≤xnumel)传入至triton_config_reduction进行配置生成

2、根据RedutionHint提示进行配置选择

- INNER： 只保留第一个配置（即最小 XBLOCK）
- OUTER： 只保留最后一个配置（即最大 XBLOCK）
- OUTER_TINY ：
  - rnumel <= 256 时，XBLOCK = 2 * (256 // rnumel)
  - 否则 XBLOCK = 1（使用最小 XBLOCK）

3、删除配置中的rblock

https://github.com/pytorch/pytorch/blob/main/torch/_inductor/runtime/triton_heuristics.py

max_autotune启发式调优

启用 max_autotune 选项时，会根据预定义的配置生成一组可能的内核配置，如果 max_autotune_gemm_search_space 设置为“EXHAUSTIVE”，就生成所有可能的配置组合，否则使用高效的预定义配置。filtered_configs函数会根据实际的矩阵大小和硬件再进行筛选，最后对筛选后的配置进行基准测试，选择性能最优的配置。

```
mm_kernel_configs = (
    [
        {"config": (32, 32, 16, 1, 2), "cond": True},
        {"config": (32, 32, 128, 2, 4), "cond": torch.version.hip is None},
        {"config": (32, 64, 32, 5, 8), "cond": True},
        {"config": (64, 32, 32, 5, 8), "cond": True},
        {"config": (64, 32, 128, 5, 4), "cond": True},
        {"config": (64, 64, 16, 2, 4), "cond": True},
        {"config": (64, 64, 32, 2, 4), "cond": True},
        {"config": (64, 64, 64, 3, 8), "cond": True},
        {"config": (64, 64, 128, 5, 4), "cond": True},
        {"config": (64, 128, 32, 3, 4), "cond": True},
        {"config": (64, 128, 32, 4, 8), "cond": True},
        {"config": (64, 128, 64, 3, 4), "cond": True},
        {"config": (64, 128, 128, 4, 4), "cond": True},
        {"config": (128, 64, 32, 3, 4), "cond": True},
        {"config": (128, 64, 32, 4, 8), "cond": True},
        {"config": (128, 128, 32, 2, 8), "cond": True},
        {"config": (128, 128, 32, 3, 4), "cond": True},
        {"config": (128, 128, 64, 3, 4), "cond": True},
        {"config": (128, 128, 64, 5, 8), "cond": True},
    ]
    if inductor_config.max_autotune_gemm_search_space != "EXHAUSTIVE"
    else [
        {"config": (BLOCK_M, BLOCK_N, BLOCK_K, num_stages, num_warps), "cond": True}
        for BLOCK_M, BLOCK_N, BLOCK_K in itertools.product(
            [16, 32, 64, 128, 256], repeat=3
        )
        for num_stages in [1, 2, 3, 4, 5]
        for num_warps in [2, 4, 8]
    ]
)
```

max_autotune启发式调优

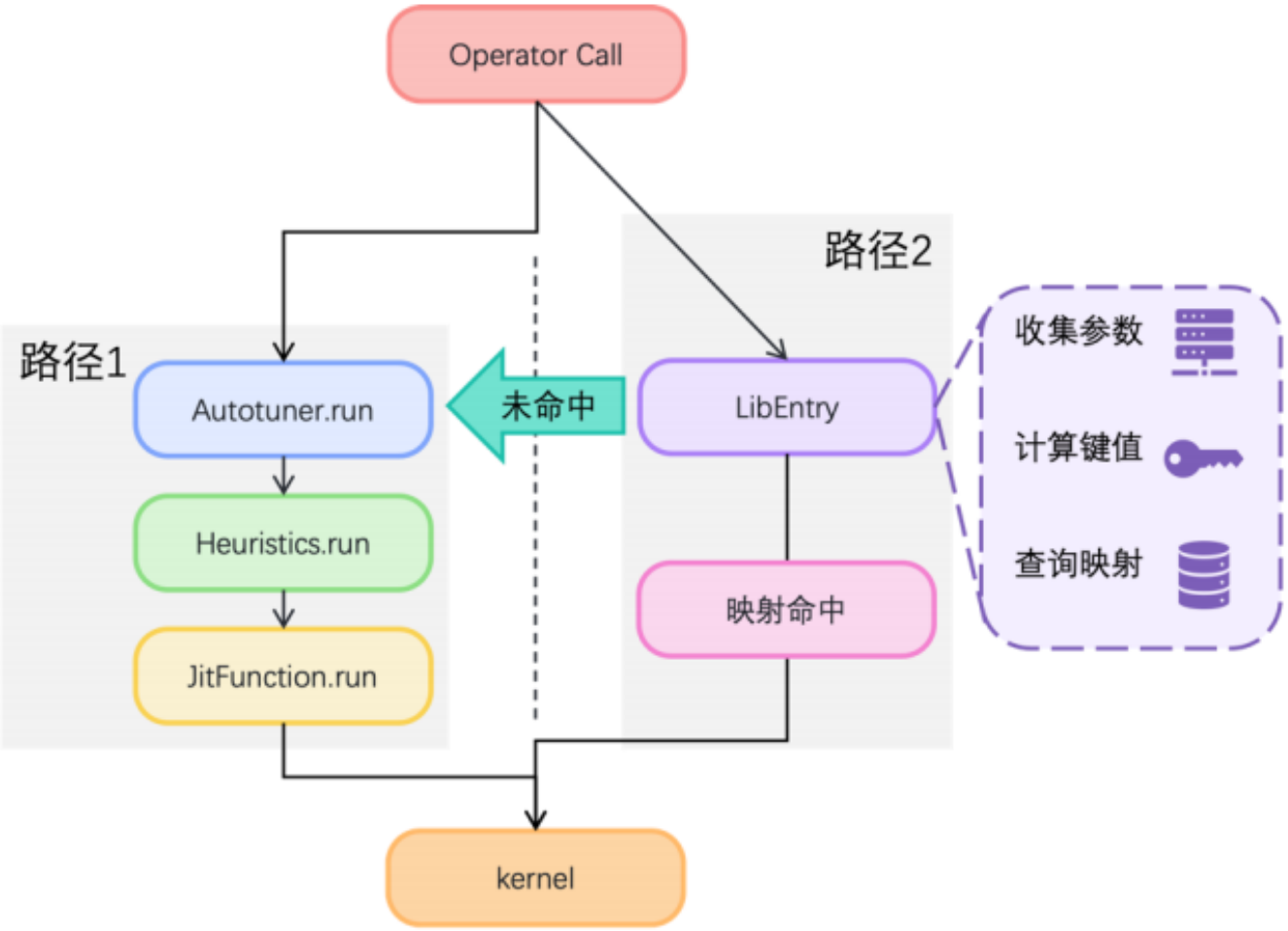
filtered_configs调整规则：

1. **参数调整:**计算并确保矩阵的大小 (m, n, k) 不小于最小的块大小，并调整为大于等于16或32 (对于int8 张量)，且是 2 的幂。
2. **配置筛选:**遍历所有给定的配置，确保块大小 (block_m, block_n, block_k) 不小于最小的块大小，并调整 num_warps以适应矩阵大小。
3. **平台特定调整**： 对于 ROCm 环境，确保块大小 block_m 和 block_n 是特定值的倍数 (如 matrix_instr_nonkdim)，并根据平台要求调整配置。
4. **去重:**利用 used 集合去重，避免生成重复的配置。

https://github.com/pytorch/pytorch/tree/main/torch/_inductor/kernel

FlagGems调优方式

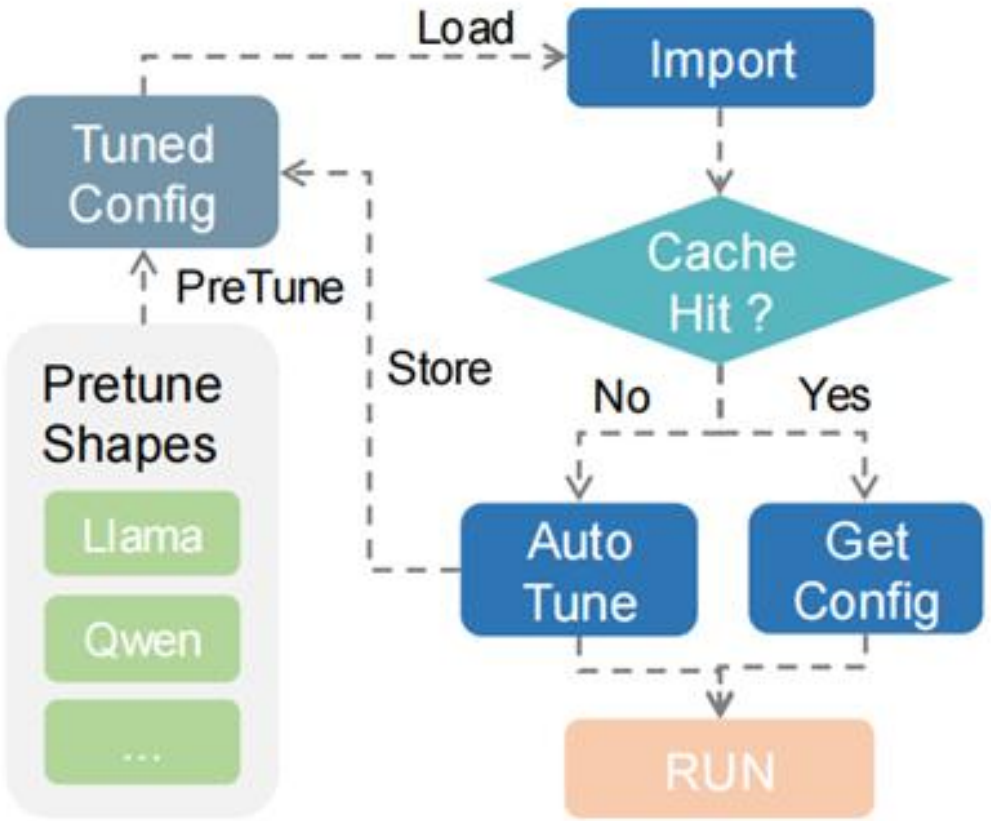
编译器在执行函数时需要调用所有装饰器的执行序列， kernel算子前的Autotuner、Heuristics和JitFunction等复杂的包装会为运行时带来较大的时间开销。为了尽可能压缩运行时的过程，FlagGems设计了LibEntry机制，使用独立维护的缓存来获取和调用核函数，绕过其他运行时装饰器的多层调用。



@libtuner

当@libentry需要实时接收动态输入形状时，运行时参数缓存命中率低就会降低，会频繁触发算子即时调优，严重延长推理响应时间。

@libtuner是通过离线缓存的方式把一些形状所对应的参数提前存到本地，在使用的时候可以去本地加载，避免进行实时的调优。



TileLang

TileLang: A Composable Tiled Programming Model for AI Systems

LEI WANG[§], Peking University, China
YU CHENG[§], Peking University, China
YINING SHI[§], Peking University, China
ZHENGJU TANG, Peking University, China
ZHIWEN MO, Imperial College London, United Kingdom
WENHAO XIE, Peking University, China
LINGXIAO MA, Microsoft Research, China
YUQING XIA, Microsoft Research, China
JILONG XUE, Microsoft Research, China
FAN YANG, Microsoft Research, China
ZHI YANG, Peking University, China

线程块级别并行编程

- 三层抽象设计
- 线程级编程能力
- 显示内存布局控制
- 软件流水线支持
- 多层次同步原语

- 一个专为高性能GPU/CPU内核开发设计的领域特定语言(DSL)，采用Python语法，底层基于TVM编译器基础设施构建
- 支持多种硬件后端

  CUDA GPUs：H100，A100…

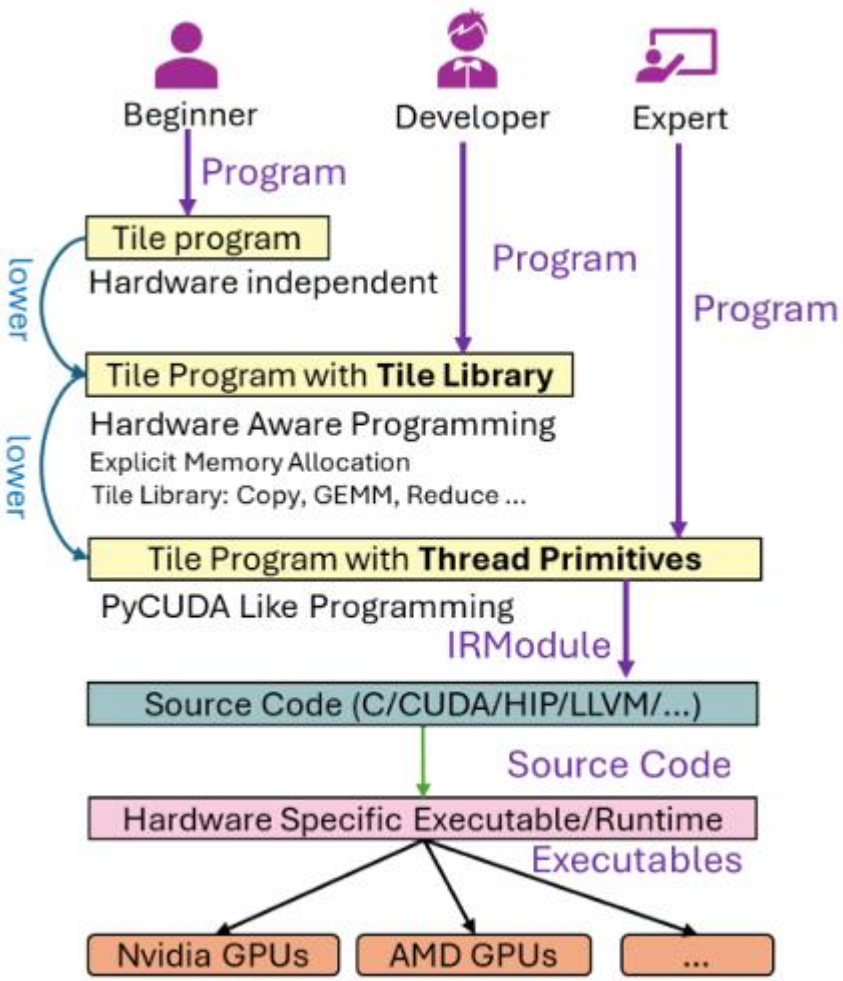
  ROCm GPUs：MI300，MI250…

  国产芯片：昇腾 910B

- **2025 年1月20日：github开源**
- **2025年9月：DeepSeek v3.2采用其开发算子**

TileLang
TILE LANGUAGE

TileLang



三级编程抽象

- Level 1: 自动调度层

```python
@T.prim_func
def level1_add(A: T.Tensor((M, N), "float16"),
               B: T.Tensor((M, N), "float16")):
    for i, j in T.grid(M, N):
        B[i, j] = A[i, j] * 2.0
```

- Level 2：块级编程层

```python
@T.prim_func
def level2_copy(A: T.Tensor((M, N), "float16"),
                B: T.Tensor((M, N), "float16")):
    with T.Kernel(T.ceildiv(N, 128), T.ceildiv(M, 128), threads=128) as (bx, by)
        A_shared = T.alloc_shared((128, 128), "float16")

        for i, j in T.Parallel(128, 128):
            A_shared[i, j] = A[by * 128 + i, bx * 128 + j]
            B[by * 128 + i, bx * 128 + j] = A_shared[i, j] * 2.0
```

- Level 3: 线程级控制层

```python
@T.prim_func
def level3_reduce(A: T.Tensor((N,), "float16"),
                  result: T.Tensor((1,), "float16")):
    with T.Kernel(1, threads=128) as _:
        tid = T.get_thread_binding(0)  # threadIdx.x
        local_sum = T.alloc_local((1,), "float")

        local_sum[0] = A[tid] if tid < N else 0.0

        # Warp-level reduction
        for offset in [16, 8, 4, 2, 1]:
            local_sum[0] += T.shfl_down(local_sum[0], offset)

        if tid == 0:
            result[0] = local_sum[0]
```

TileLang

多层次同步原语

- 线程块同步：T.sync_threads支持整个线程块以及部分线程块同步
- 线程束Warp同步：支持T.shfl_xor、T.shfl_down、 T.shfl_up等shuffle操作
- 网格级同步：T.sync_grid、T.sync_global
- Barrier：T.mbarrier_wait_parity、T.mbarrier_arrive、T.mbarrier_expect_tx

多层次同步原语

- 共享内存：T.alloc_shared、T.alloc_barrier
- 寄存器：T.alloc_fragment、T.alloc_var
- 本地内存：T.alloc_local

Triton Matrix Mult

```
import torch
import triton
import triton.language as tl
@triton.jit
def simple_matmul_kernel(
    a_ptr, b_ptr, c_ptr,
    M, N, K,
    stride_am, stride_ak,
    stride_bk, stride_bn,
    stride_cm, stride_cn,
    BLOCK_SIZE_M: tl.constexpr,
    BLOCK_SIZE_N: tl.constexpr,
    BLOCK_SIZE_K: tl.constexpr,
):
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)
    offs_m = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
    offs_n = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
    offs_k = tl.arange(0, BLOCK_SIZE_K)
    a_ptrs = a_ptr + offs_m[:, None] * stride_am + offs_k[None, :] * stride_ak
    b_ptrs = b_ptr + offs_k[:, None] * stride_bk + offs_n[None, :] * stride_bn
    accumulator = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=tl.float32)
```

```
for k in range(0, K, BLOCK_SIZE_K):
    a = tl.load(a_ptrs)
    b = tl.load(b_ptrs)
    accumulator += tl.dot(a, b)
    a_ptrs += BLOCK_SIZE_K * stride_ak
    b_ptrs += BLOCK_SIZE_K * stride_bk
c = accumulator.to(tl.float16)
c_ptrs = c_ptr + offs_m[:, None] * stride_cm + offs_n[None, :] * stride_cn
tl.store(c_ptrs, c)
```

上面两段代码展示了Triton矩阵乘内核的实现，可以看到代码中包含大量的块偏移计算和地址掩码计算，编写复杂且容易出错，对初学者不友好，代码抽象层次低

Triton Matrix Mult

下面是对应wrapper 函数，主要对输入进行检查并启动内核

```
def simple_matmul(a, b, block_m=128, block_n=128, block_k=32):
    assert a.shape[1] == b.shape[0], "矩阵维度不匹配"
    M, K = a.shape
    K, N = b.shape
    c = torch.empty((M, N), device=a.device, dtype=a.dtype)
    grid = (triton.cdiv(M, block_m), triton.cdiv(N, block_n))
    simple_matmul_kernel[grid](
        a, b, c, M, N, K,
        a.stride(0), a.stride(1),
        b.stride(0), b.stride(1),
        c.stride(0), c.stride(1),
        BLOCK_SIZE_M=block_m,
        BLOCK_SIZE_N=block_n,
        BLOCK_SIZE_K=block_k,
    )

    return c
```

主函数中实例化输入，调用内核，并与pytorch结果对比

```
def main():
    M, N, K = 1024, 1024, 1024
    block_m, block_n, block_k = 128, 128, 32
    a = torch.randn(M, K, device='cuda', dtype=torch.float16)
    b = torch.randn(K, N, device='cuda', dtype=torch.float16)
    c_triton = simple_matmul(a, b, block_m, block_n, block_k)
    c_ref = torch.matmul(a, b)
    print("Triton结果:")
    print(c_triton[:10, :10])
    print("PyTorch参考结果:")
    print(c_ref[:10, :10])
    try:
        torch.testing.assert_close(c_triton, c_ref, rtol=1e-2, atol=1e-2)
        print("结果验证通过!")
    except AssertionError as e:
        print(f"结果验证失败: {e}")
if __name__ == "__main__":
    main()
```

TileLang Matrix Mult

内核函数实现

```
import tilelang
import tilelang.language as T
@tilelang.jit(out_idx=[-1])
def matmul(M, N, K, block_M, block_N, block_K, dtype="float16", accum_dtype="float"):
    @T.prim_func
    def gemm(
            A: T.Tensor((M, K), dtype),
            B: T.Tensor((K, N), dtype),
            C: T.Tensor((M, N), dtype),
    ):
        with T.Kernel(T.ceildiv(N, block_N), T.ceildiv(M, block_M), threads=128) as (bx, by):
            A_shared = T.alloc_shared((block_M, block_K), dtype)
            B_shared = T.alloc_shared((block_K, block_N), dtype)
            C_local = T.alloc_fragment((block_M, block_N), accum_dtype)
            T.clear(C_local)
            for k in T.Pipelined(T.ceildiv(K, block_K), num_stages=3):
                T.copy(A[by * block_M, k * block_K], A_shared)
                T.copy(B[k * block_K, bx * block_N], B_shared)
                T.gemm(A_shared, B_shared, C_local)

            T.copy(C_local, C[by * block_M, bx * block_N])
    return gemm
```

对比Triton内核实现，可以看到TileLang的内核实现更为简洁，抽象层次更高，用户只需用极少的语句表达正确的处理逻辑即可

TileLang Matrix Mult

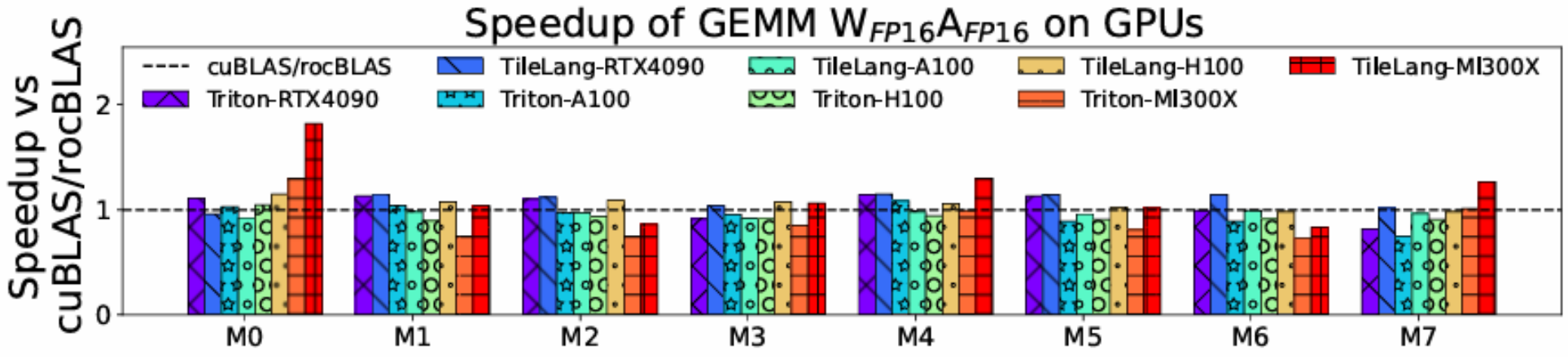
Wrapper函数实现

```
def main():
    kernel = matmul(1024, 1024, 1024, 128, 128, 32)
    import torch
    a = torch.randn(1024, 1024).cuda().half()
    b = torch.randn(1024, 1024).cuda().half()
    c = kernel(a, b)
    ref_c = a @ b
    print("c:")
    print(c)
    print("ref_c:")
    print(ref_c)
    torch.testing.assert_close(c, ref_c, rtol=1e-2, atol=1e-2)
    print("All check passed.")
    # Get CUDA Source
    print("CUDA Source:")
    print(kernel.get_kernel_source())

if __name__ == "__main__":
    main()
```

Wrapper函数实现也更简洁，不需要复杂的边界检查，只需像使用函数一样，传递符合对应的变量即可，更多的检查细节交给编译器进行检查

Matrix Mult性能对比



官方测试数据中，不同尺寸的矩阵在N卡上TileLang和Triton都接近于手写cuBLAS算子，在ROCm GPU上，TileLang实现的性能要比cuBLAS这种手写算子库和Triton优秀，考虑到TileLang内核代码实现要比Triton代码更简洁易上手，综合看来，TileLang性能要比Triton优秀得多。

自动调优对比

而对于TileLang而言，其config配置显得更智能，通过itertools包自动生成对应参数的笛卡尔积，后续可针对生成的列表从中筛选符合条件的

```
import itertools
def get_config():
        BLOCK_M = [1, 2, 4, 8,16, 32, 64, 128]
        reduce_threads = [4, 8, 16, 32, 64, 128, 256]
        _configs = list(itertools.product( BLOCK_M, reduce_threads，  ))
        configs = []
        for c in _configs :#config中也可以通过一些代码逻辑对参数进行约束
                if (c[0] * c[1] <=1024):
                        configs.append({
                                "BLOCK_M": c[0],
                                "reduce_threads": c[1],          })
        return configs
@tilelang.autotune(
        configs= get_config(),
        #调用getconfig函数.
        warmup= 50,
        rep = 100,)
```

其他 DSL

Triton Confernece 2025上 Meta 团队分享了两个DSL

- Helion：专注于高层 DSL 的抽象表达
- TLX：探索更细粒度的设备控制

OpenAI也引入了Triton实验性特性

- Gluon：可直接操作硬件底层接口

NVIDIA CUDA 13.1推出了全新的编程模型cuTile

- Triton也开始尝试接入Triton CUDA Tile IR Backend

智源研究院 FlagTree 团队推出了TLE (Triton Language Extensions)，创新的三层协同 DSL 扩展

其他 DSL

TileLang：https://github.com/tile-ai/tilelang

Helion：https://github.com/pytorch/helion

TLX：https://github.com/facebookexperimental/triton/tree/main/third_party/tlx

Gluon：https://github.com/triton-lang/triton/tree/main/python/triton/experimental/gluon

cuTile：https://github.com/nvidia/cutile-python

Triton-to-tile-IR：https://github.com/triton-lang/Triton-to-tile-IR

TLE：https://github.com/flagos-ai/FlagTree/wiki/TLE

课程总结

1、掌握Triton语法及简单Triton算子的编写

安装Triton Puzzles Lite项目，分析并运行Demo1-4的示例，掌握Triton常用API用法

尝试编写Puzzle1至Puzzle5的算子并运行

（进阶）尝试编写Puzzle6至Puzzle12的算子并运行

2、掌握Triton算子的关键参数调优使用方法

以简单的矩阵乘算子为例，结合参数配置打印调试选项，尝试运行时传参、@triton.autotune、@triton.heuristics等方式对算子中关键参数调整对算子性能的影响

（进阶）了解PyTorch Inductor针对各类别算子的启发式调优算法，并尝试进行优化改进；借鉴FlagGems关键参数调优的优化，思考其他优化改进方法

（进阶）了解Triton项目的Gluon语言，以及TileLang语言等，对比差别

第四讲准备

Install from source

```
git clone https://github.com/triton-lang/triton.git
cd triton

pip install -r python/requirements.txt # build-time dependencies
pip install -e .
```

Or with a virtualenv:

```
git clone https://github.com/triton-lang/triton.git
cd triton

python -m venv .venv --prompt triton
source .venv/bin/activate

pip install -r python/requirements.txt # build-time dependencies
pip install -e .
```

Triton

https://github.com/triton-lang/triton